IA et Jeux

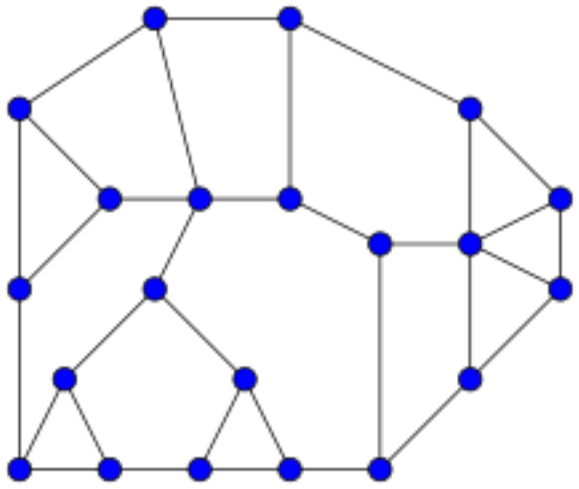
Cours 2: Algorithmique et recherche heuristique

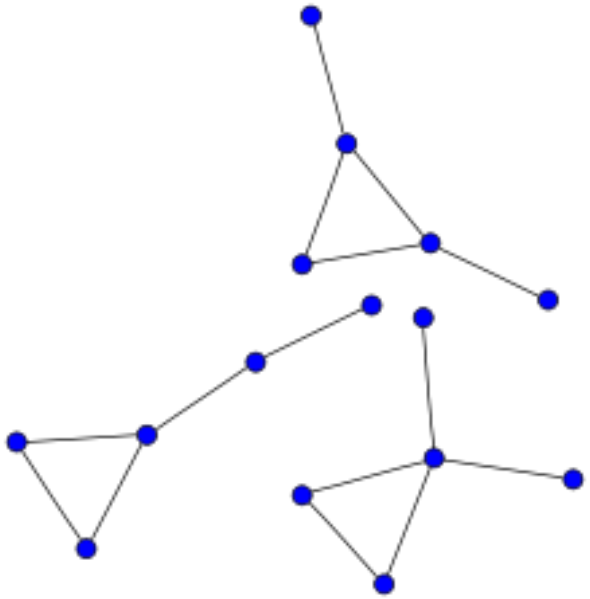
Stephane Ayache

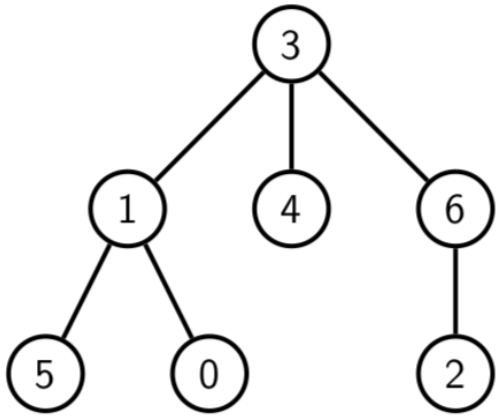
Polytech Marseille - AMU

Retour sur les arbres

- Un arbre est un graphe *(S, A)*
  - Non orienté, sans cycle, et connexe
  - S est un ensemble de Sommets
  - A un ensemble d'arêtes $\subset S \times S$
  - Un seul élément de S est la racine
  - Les éléments de S n'ayant pas de fils sont appelés *feuilles*
  - La profondeur (ou hauteur) d'un arbre est la distance maximale (nombre de sommets) entre la racine et une feuille
  - L'*arité* d'un sommet est son nombre de fils







Arbres n-aires

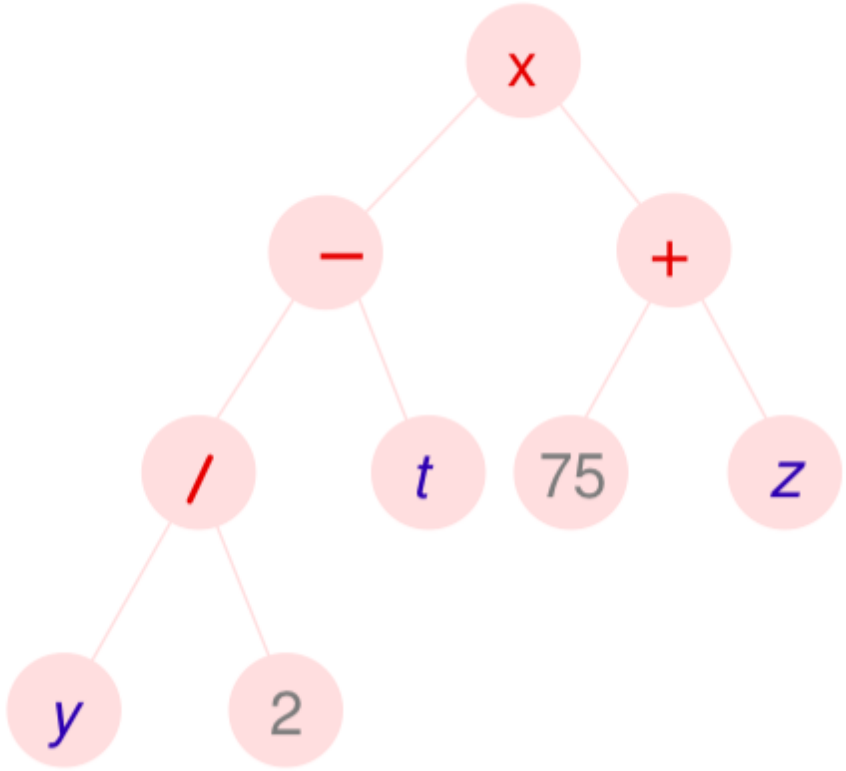
- Arbres d’arités quelconque

- Exemple : arbres syntaxiques

Arbres binaires

- Arbres d'arités ≤ 2
- Hauteur $h \leq \log_2(n)$

- Exemple : expressions arithmétiques

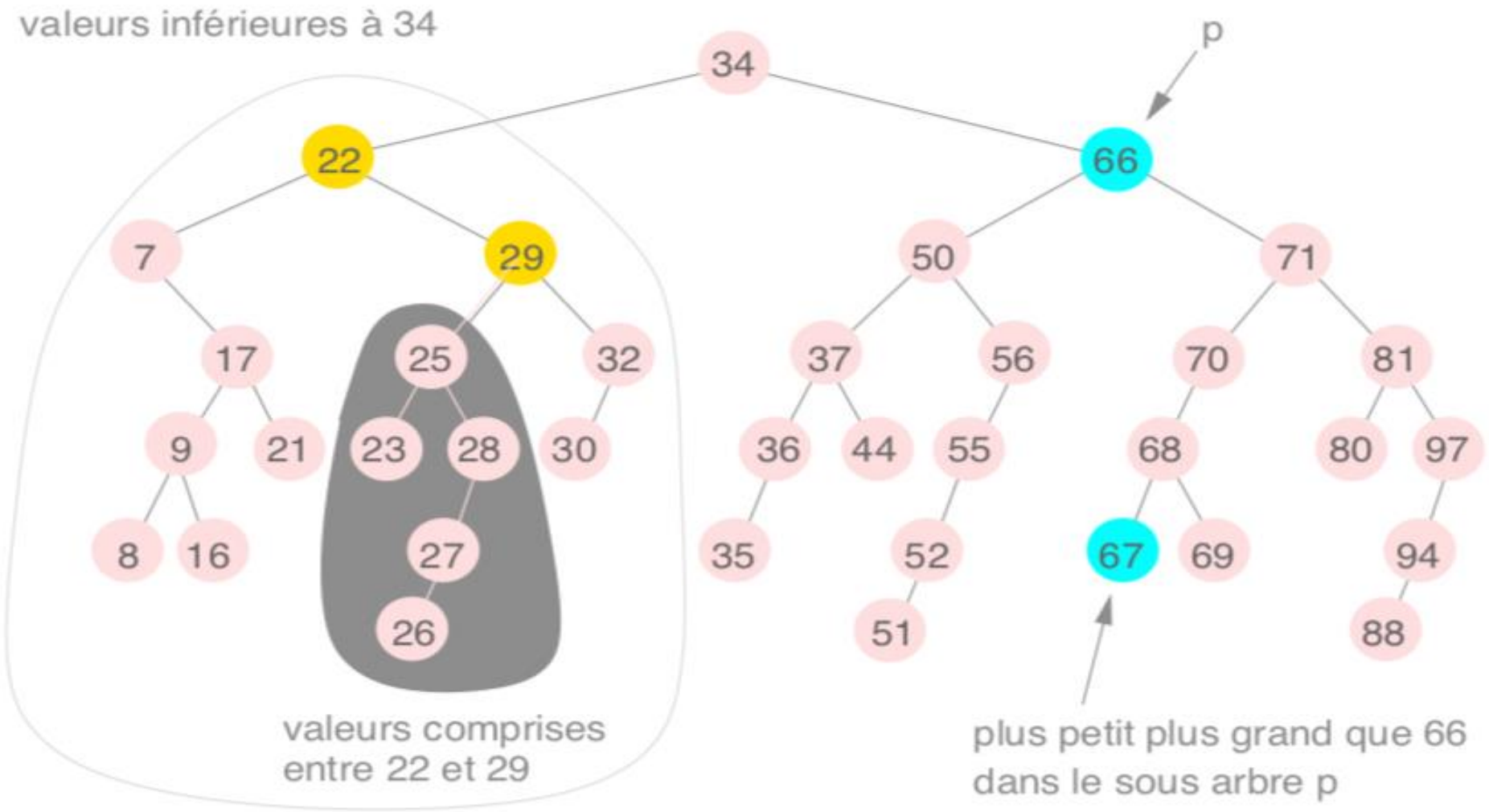


Arbres binaires de recherche

- Structure de données pour laquelle la recherche est efficace
- Si une relation d’ordre existe entre les sommets
- Les opérations *insertion* et *suppression* doivent maintenir cette propriété

- Exemple : fils gauche ≤ fils droit

Arbres binaires de recherche



Implémentation - type

- Arbre binaire

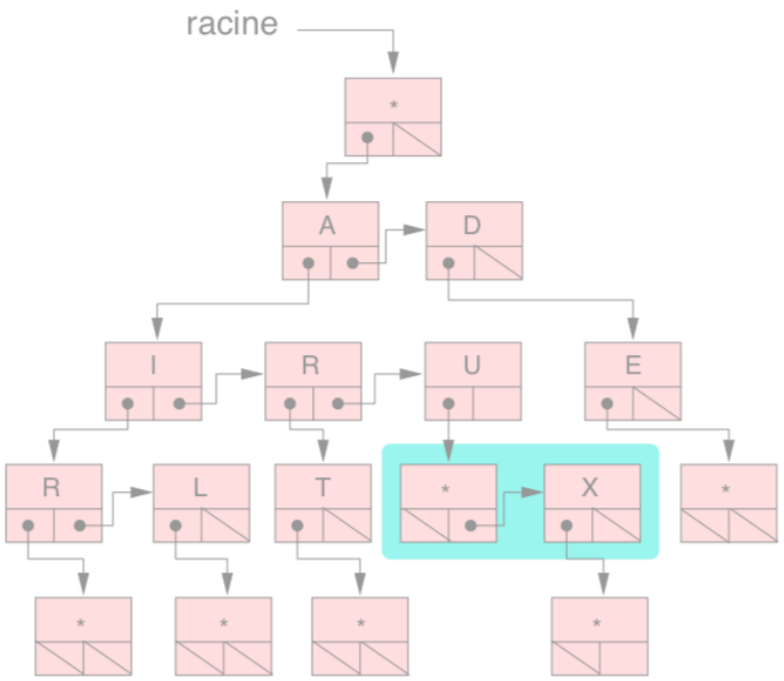
```
typedef struct noeud *ARBRE;

struct noeud
{
   VALEUR val;
   ARBRE fg, fd, pere;
};
```

- Arbre n-aire

```
typedef struct noeud * ARBREn;

struct noeud
{
   TYPE_VALEUR val;
   ARBREn fa;
   ARBREn fr;
};
```

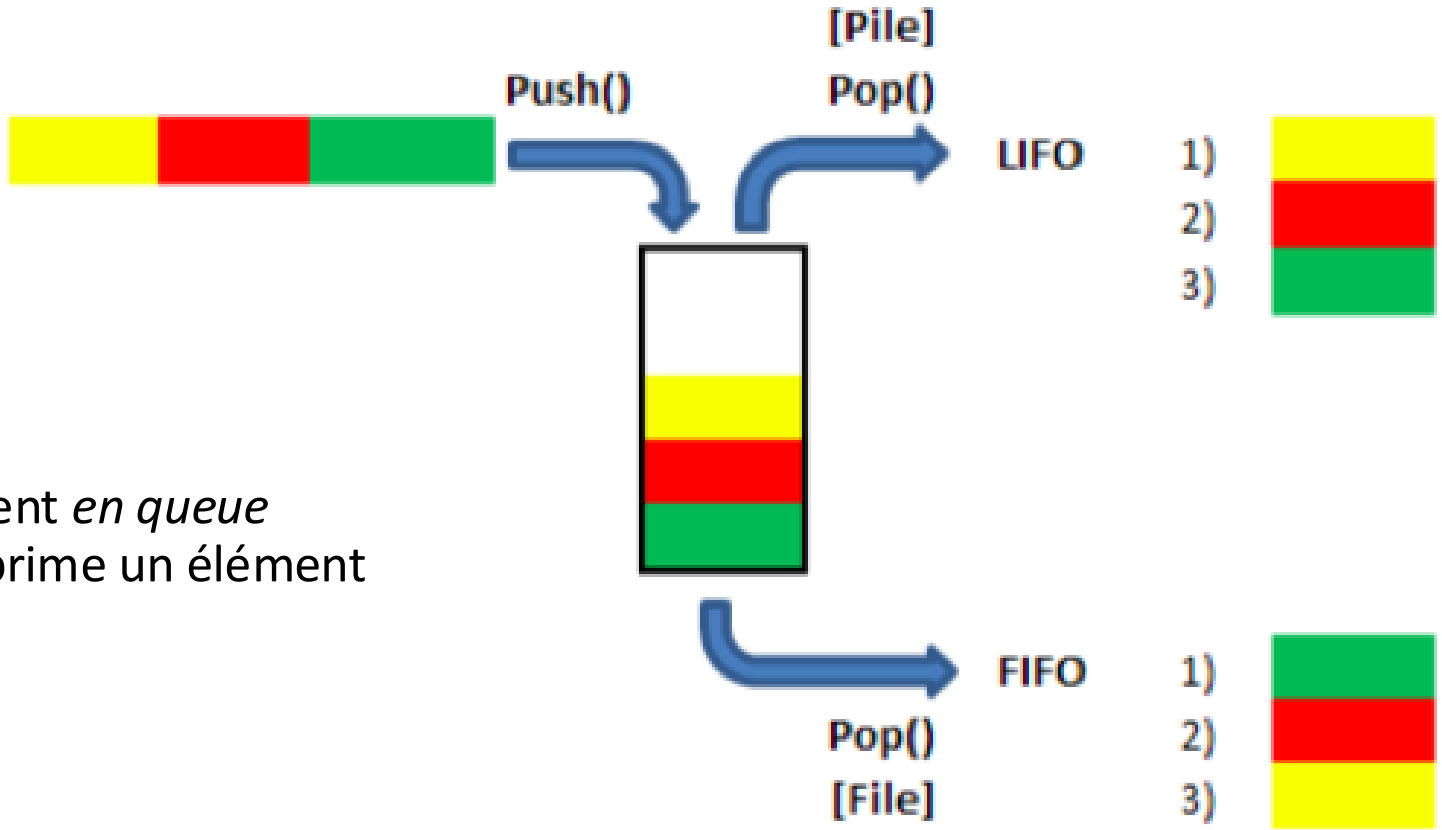


Parcours d’arbres

- Utile pour rechercher un élément dans une structure arborescente
- 2 types de parcours
  - Largeur d’abord
  - Profondeur d’abord
- Implémentation récursive ou itérative (avec listes)

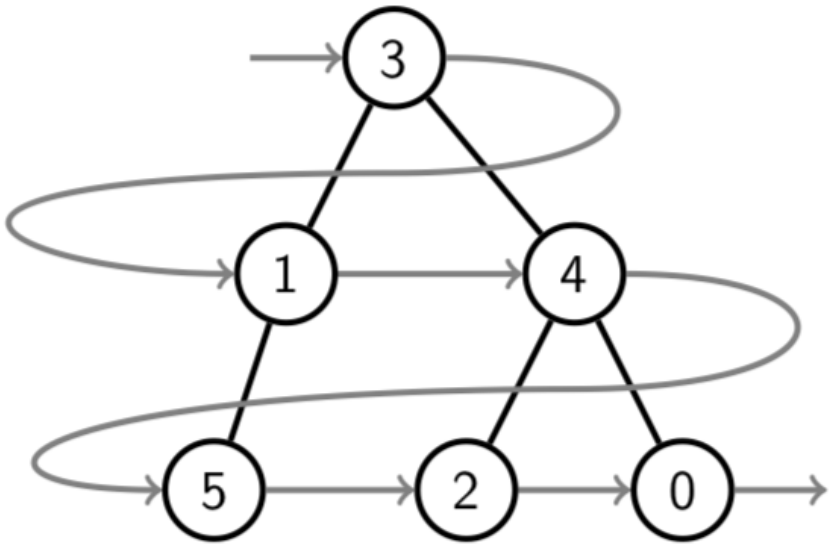
Rappel listes Files et Piles : FIFO, LIFO

- FIFO : First In, First out
  - File
- LIFO : Last In, First out
  - Pile
- 2 fonctions
  - Push() : ajoute un élément *en queue*
  - Pop() : récupère et supprime un élément
    - File → *en tête*
    - Pile → *en queue*
- Implémentation
  - Tableau (peu efficace)
  - Liste chainée (efficace)



Parcours en largeur d'abord

- *« Breadth First Search »*
- Parcours par distance croissante à la racine
- Algorithme itératif avec File

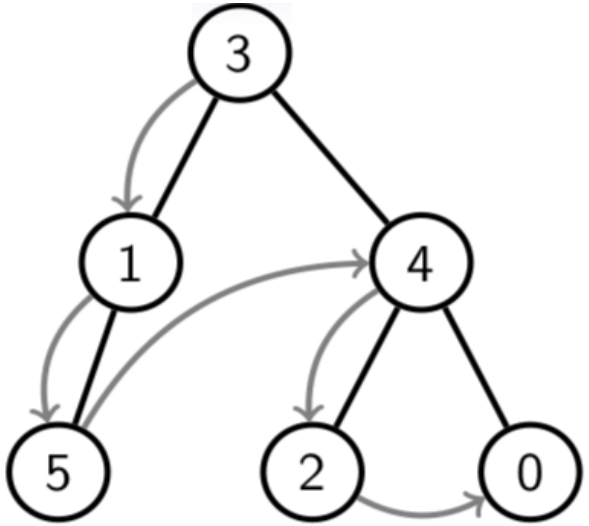



```
BFS (G, s):
    let L be File // openlist
    L.push( s )   // append in queue of list
    mark s as visited // can use a closedlist to avoid cycles

    while ( L is not empty)
        v = L.pop( ) // from head of list
        print (v)
        for all neighbours w of v in Graph G  //processing all the successors of v
            if w is not visited: L.push( w )  //Stores w in L to further visit its successors
            mark w as visited
```

Parcours en profondeur d'abord

- *« Depth First Search »*
- Parcours récursif
- Algorithme itératif avec Pile

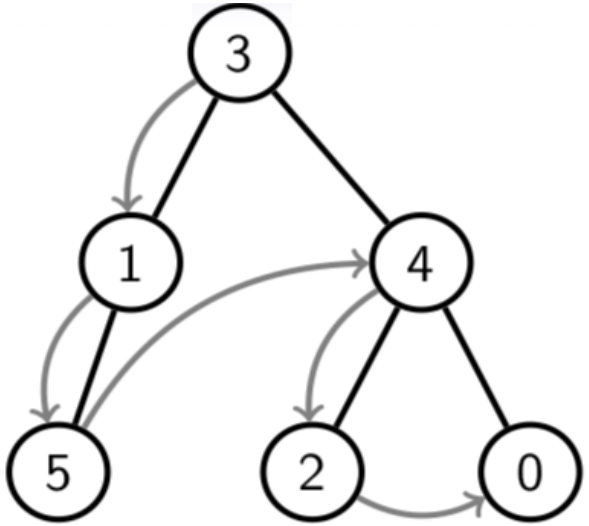



```
DFS (G, s):
    let L be Pile // openlist
    L.push( s )   // append in queue of list
    mark s as visited // can use a closedlist to avoid cycles

    while ( L is not empty)
        v = L.pop( ) // from queue of list
        print (v)
        for all neighbours w of v in Graph G  //processing all the successors of v
            if w is not visited: L.push( w )  //Stores w in Q to further visit its successors
            mark w as visited
```

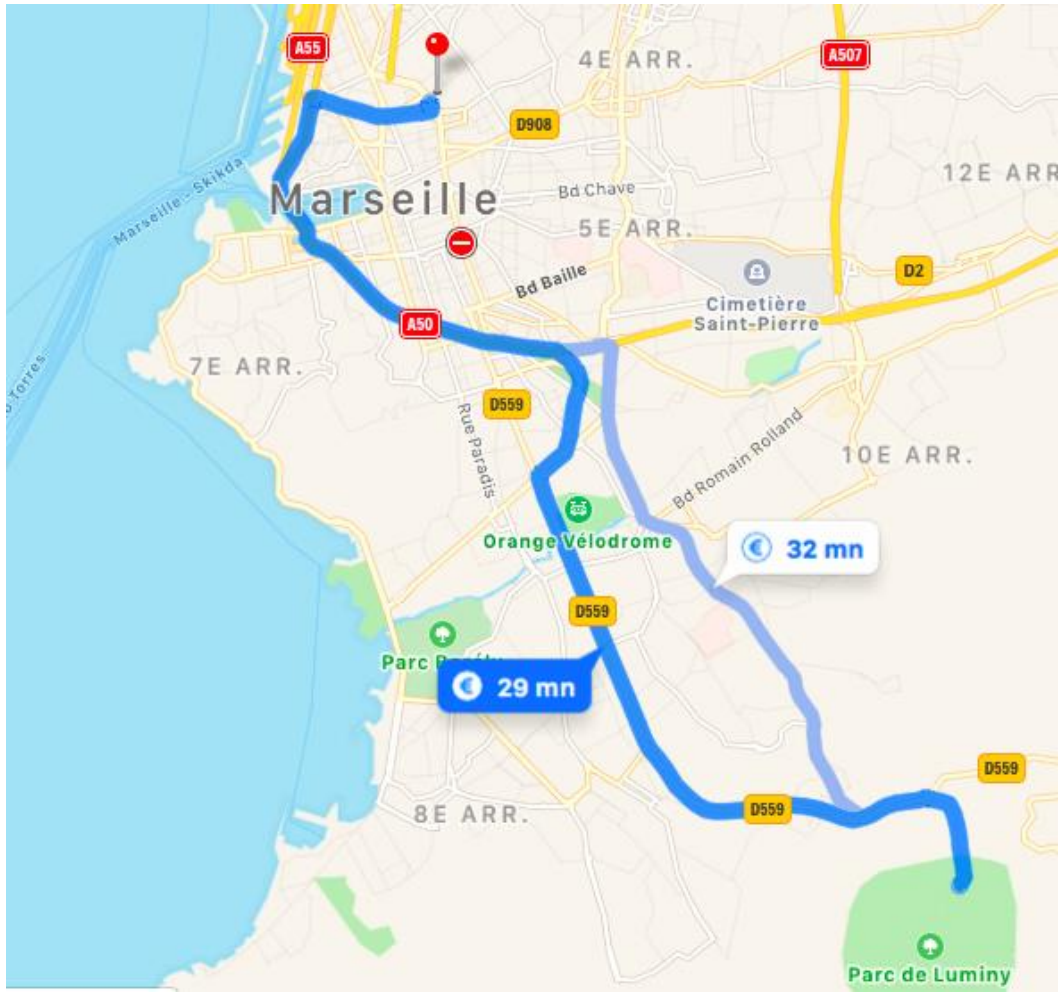
Parcours en profondeur d'abord

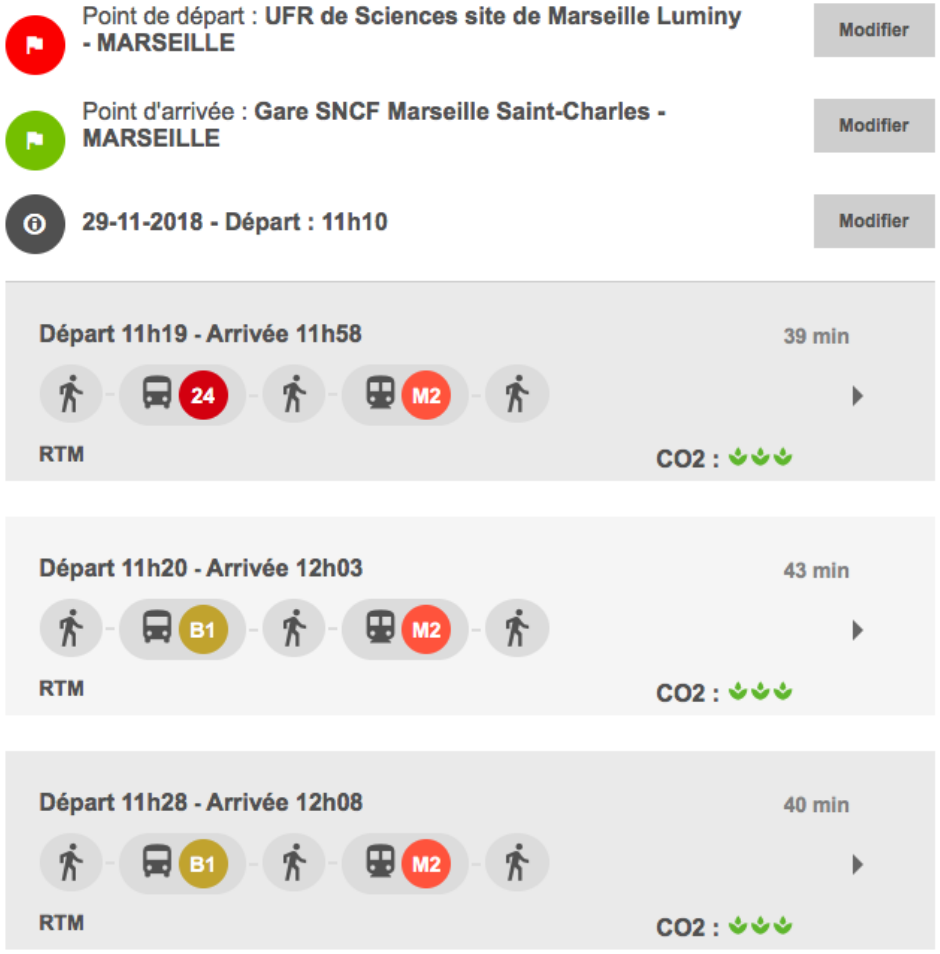
- *« Depth First Search »*
- Parcours récursif
- Algorithme récursif




```
DFS-recursive(G, s):
    mark s as visited
    for all neighbours w of s in Graph G:
        if w is not visited: DFS-recursive(G, w)
```

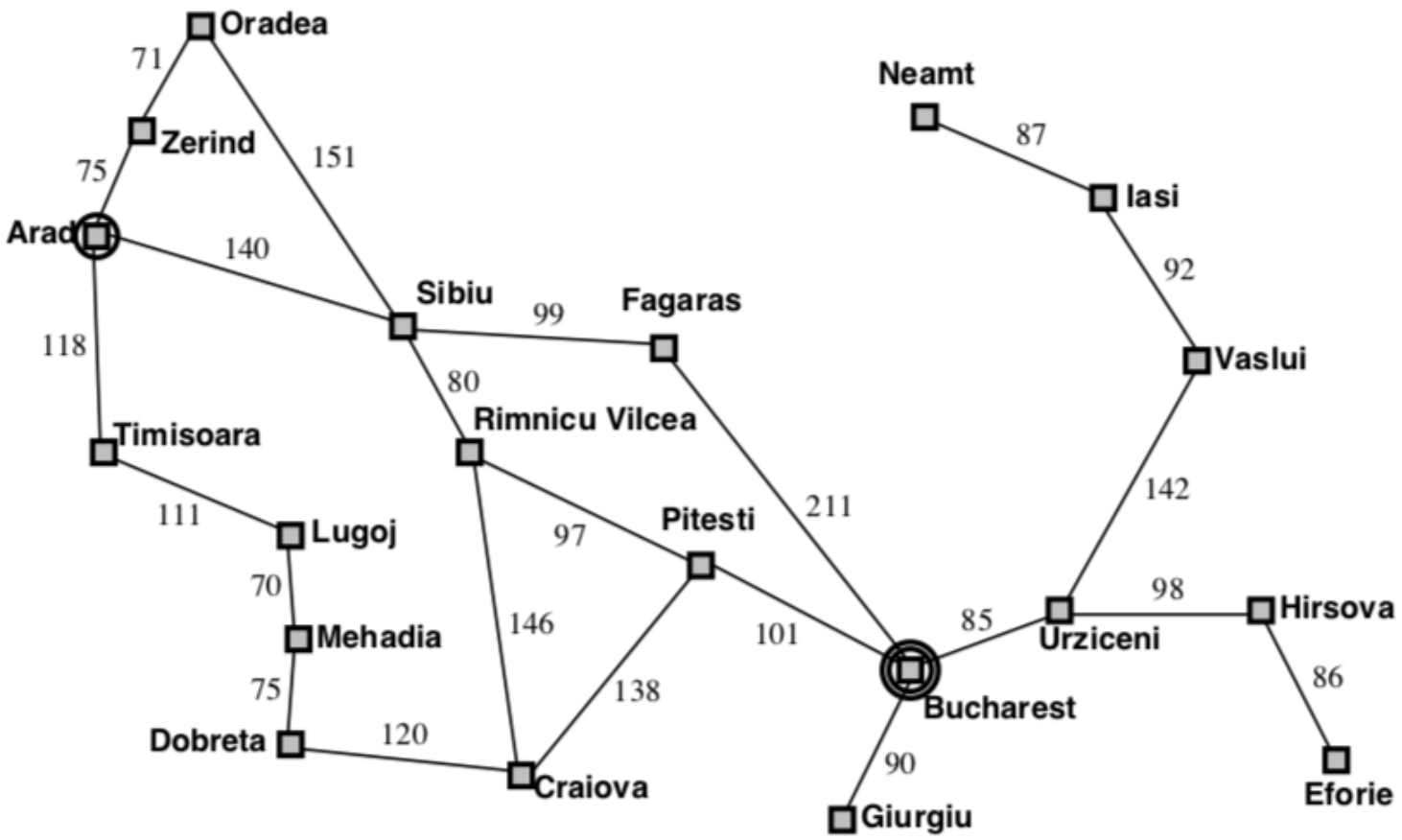
Plus court chemin dans un graphe (ou arbre)





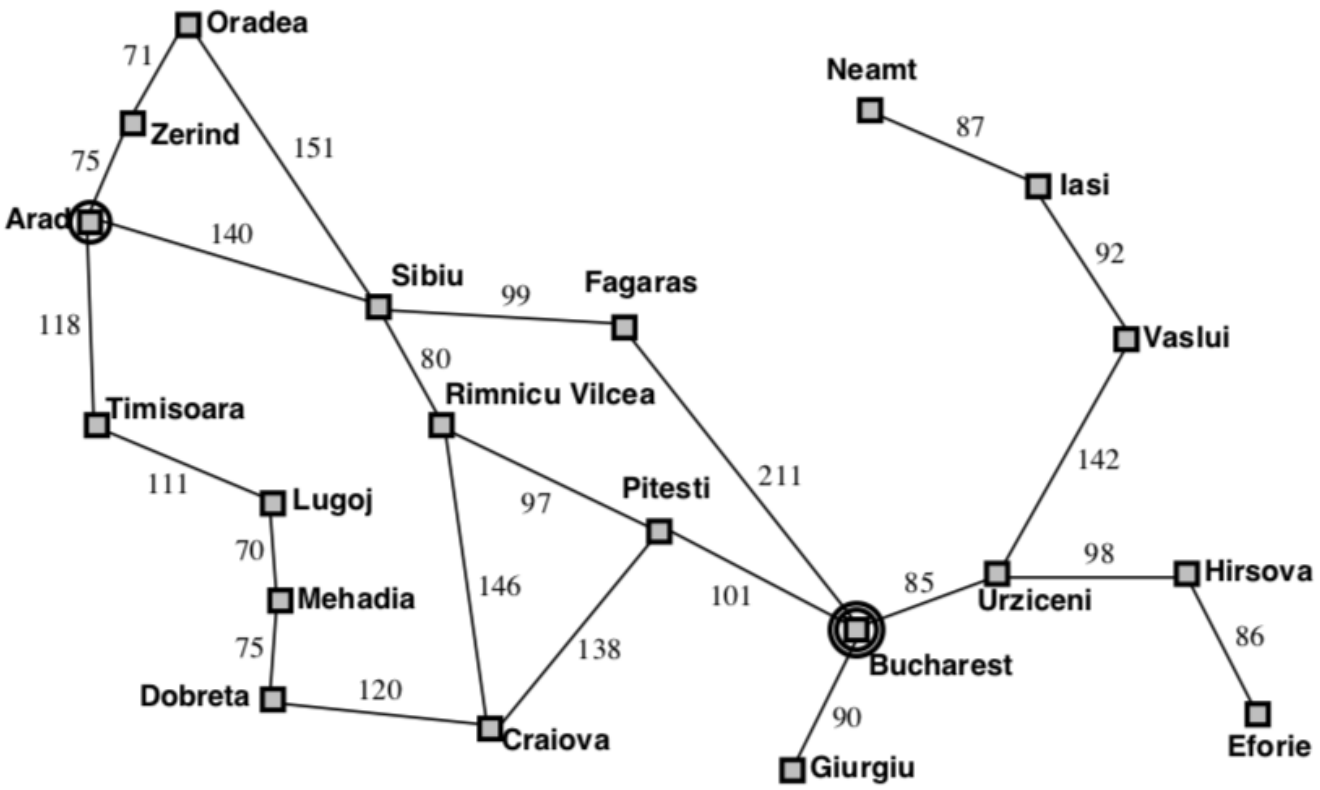
Plus court chemin dans un graphe (ou arbre)

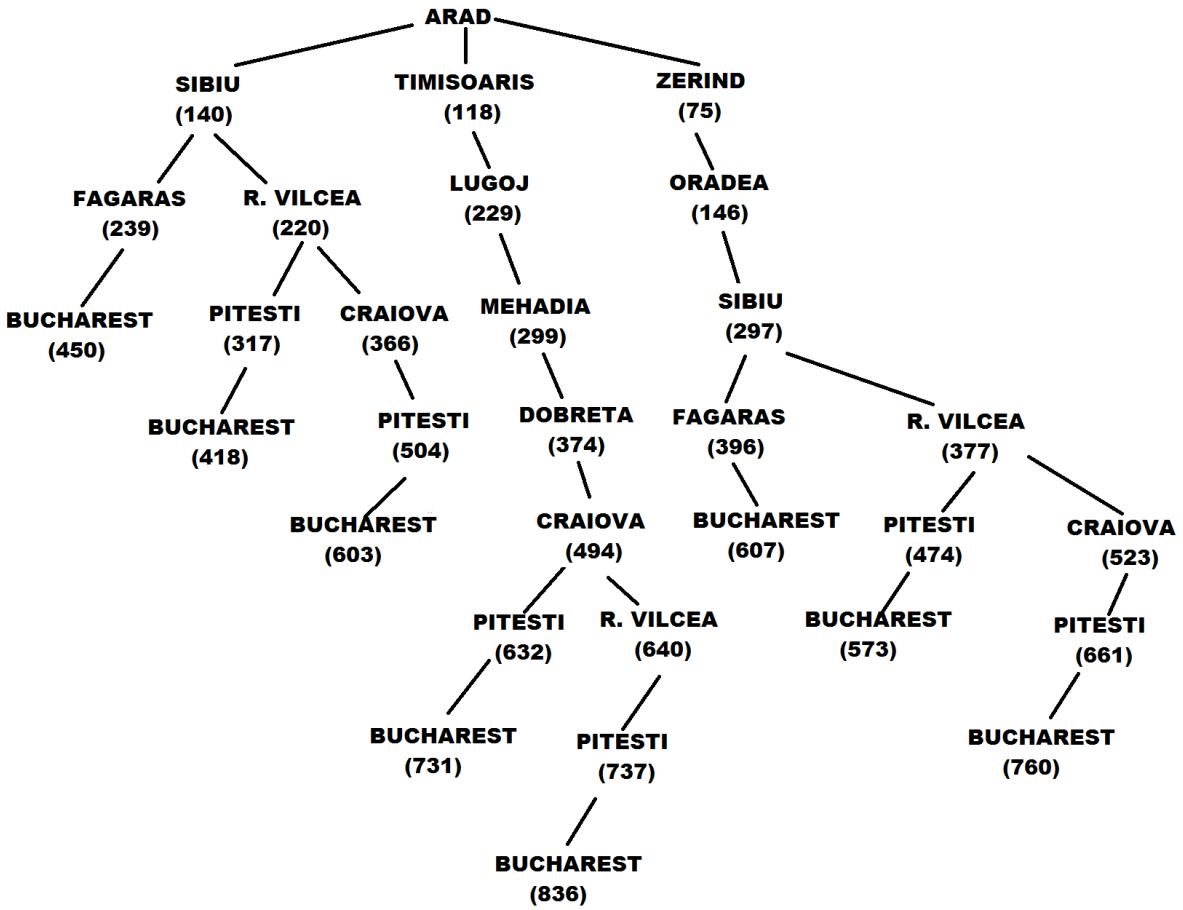
- Quel est le plus court chemin entre Arad et Bucarest ?



Plus court chemin dans un graphe (ou arbre)

- Quel est le plus court chemin entre Arad et Bucarest ?





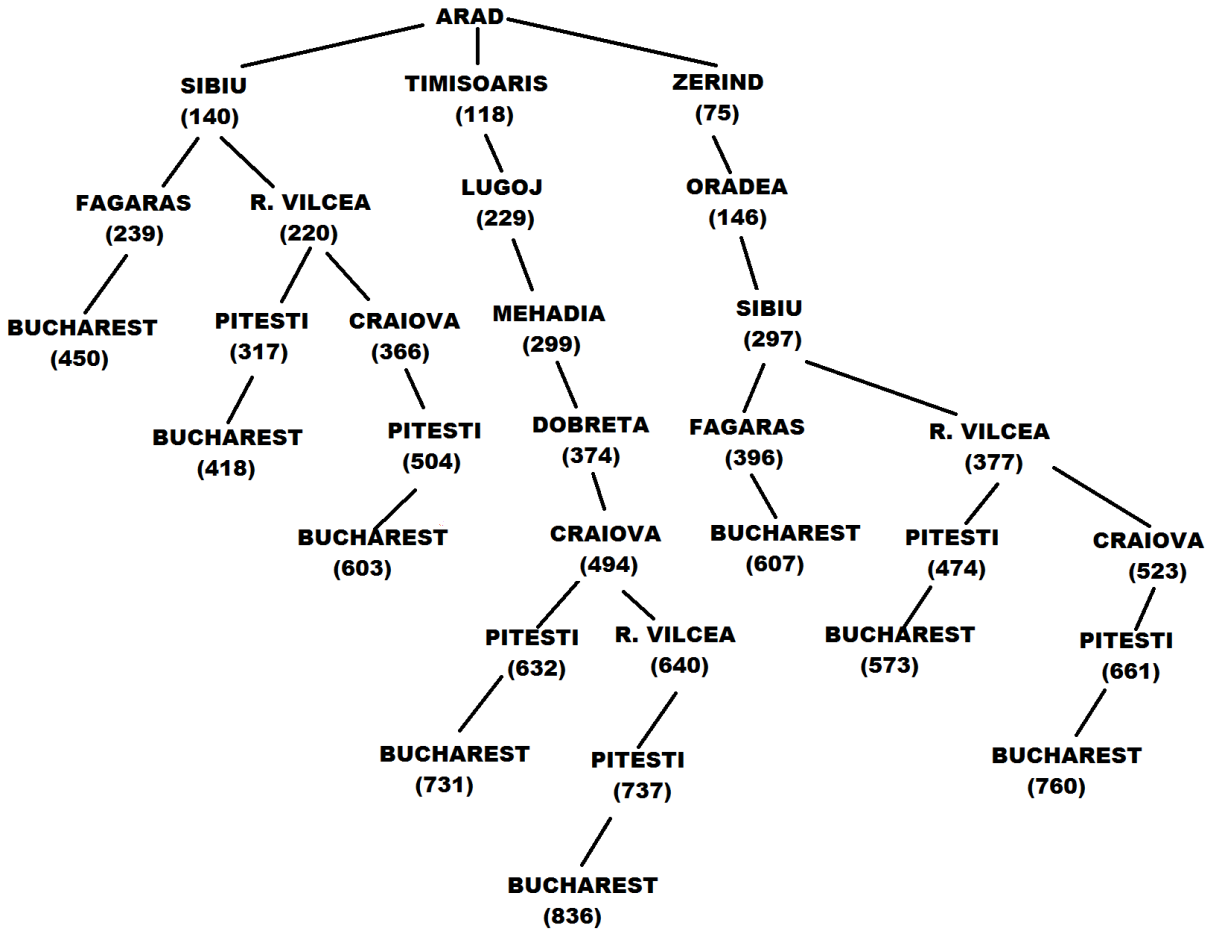
Plus court chemin dans un graphe (ou arbre)

- Quel est le plus court chemin entre Arad et Bucarest ?

  ⇒ Parcours en largeur d'abord ?

  ⇒ Parcours en profondeur d'abord ?

- Parcours exhaustif vs recherche
  - Arrêt à la première solution trouvée



L'algorithme de Dijkstra

- Trouve le plus court chemin entre deux sommets d’un graphe non orienté
- [Dijkstra, 1956]

- Adapté d’un parcours en largeur d’abord (BFS)

- Poids positifs sur les arêtes
  - Voir sinon : [Bellman-Ford], [Floyd-Warshall]

L'algorithme de Dijkstra

- Stocker une valeur temporaire *dist* (ou *coût*) par sommet
  - Correspond à la distance minimale entre la racine et un noeud
  - Tableau de taille *Card(S) o*u ajout d'un attribut *dist* dans la structure *Nœud*
  - Pour tous les nœuds, *dist* est initialisé à +∞

- A chaque itération, recherche l'élément de *dist* minimal

- Complexité grandement réduite avec une liste *à priorité*
  - Liste constamment triée selon un critère (par exemple *dist*)
  - Les opérations d'insertion et suppression préservent cette propriété
  - (aussi appelé *Tas*)

L'algorithme de Dijkstra

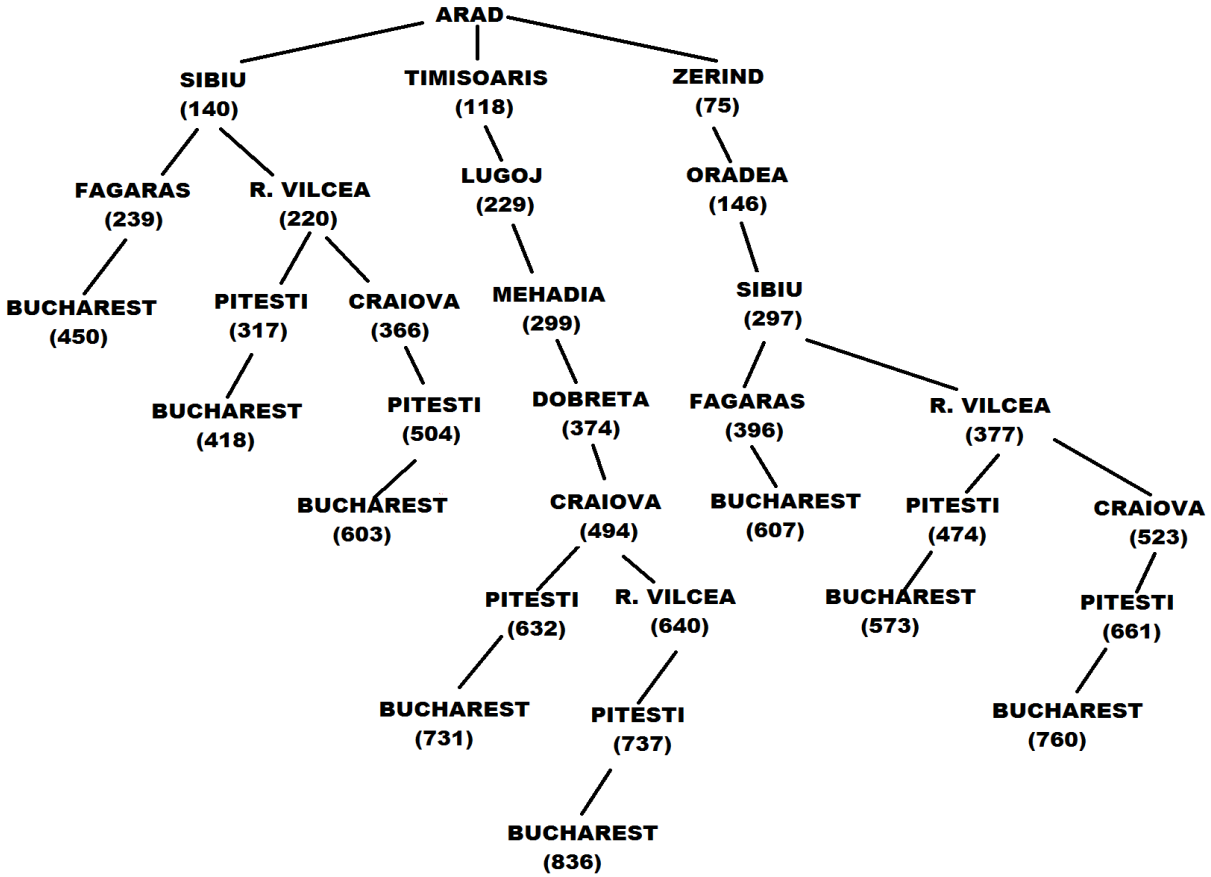
```
Dijkstra (G, s, weights):
    let L be Tas  // openlist
    L.push( s )   // append in priority queue
    mark s as visited // can use a closedlist to avoid cycles
    s.dist = 0
    while ( L is not empty)
        v = L.pop( ) // from head of list: get the minimum in priority queue
        print (v)
        for all neighbours w of v in Graph G  //processing all the successors of v
            if w is not visited and w.dist > v.dist + weight[v,w]:
                w.dist = v.dist + weight[v,w]
                L.push( w )  // If w already in L, replace it! = decrease_key
                mark w as visited
```

https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1172/handouts/search.html

Plus court chemin dans un graphe (ou arbre)

- Quel est le plus court chemin entre Arad et Bucarest ?

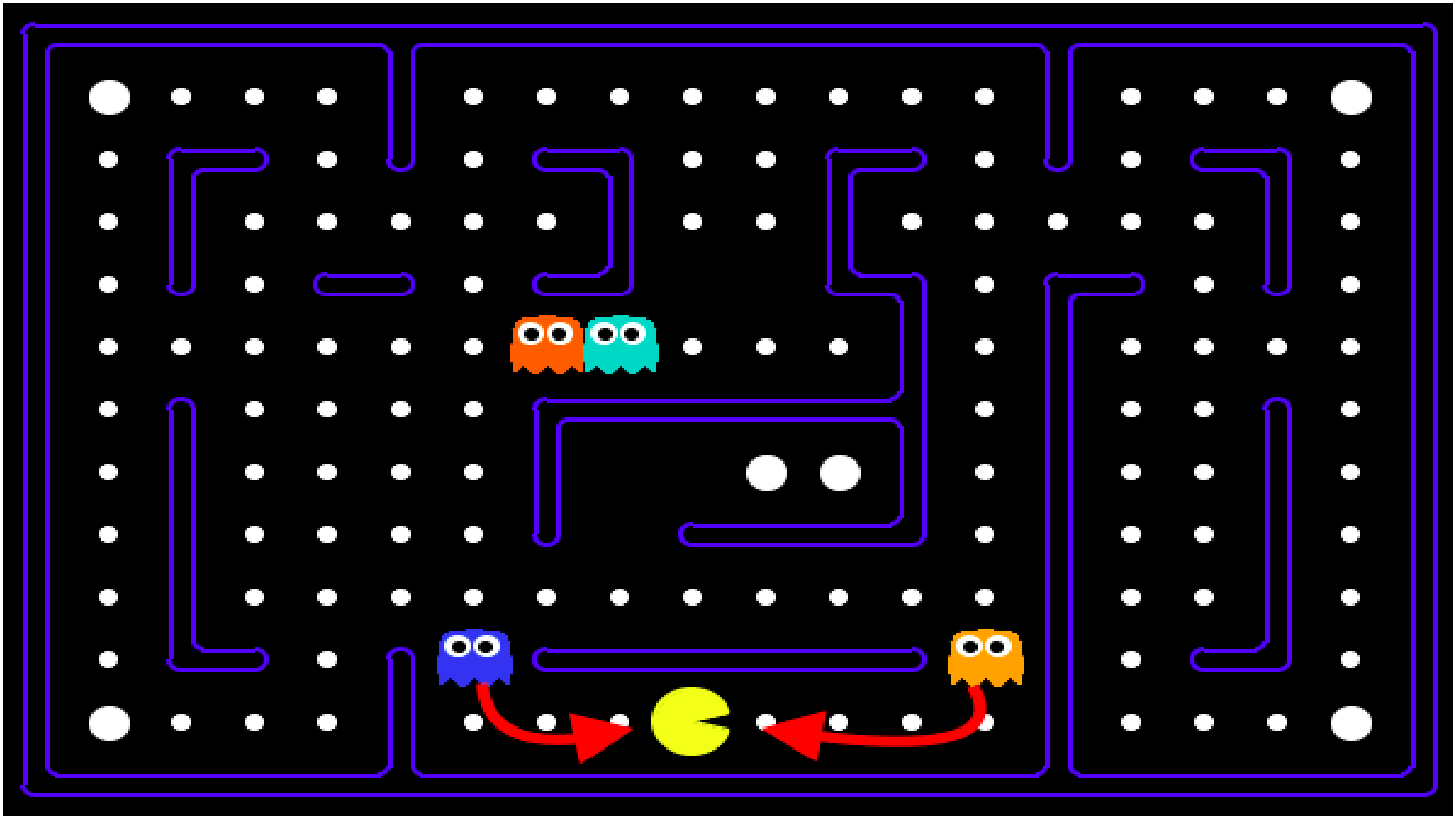
⇒ Dijkstra !



Bon ok… Et les jeux alors !?

Fortnite : Zombies pathfinding

Pacman: Ghosts pathfinding



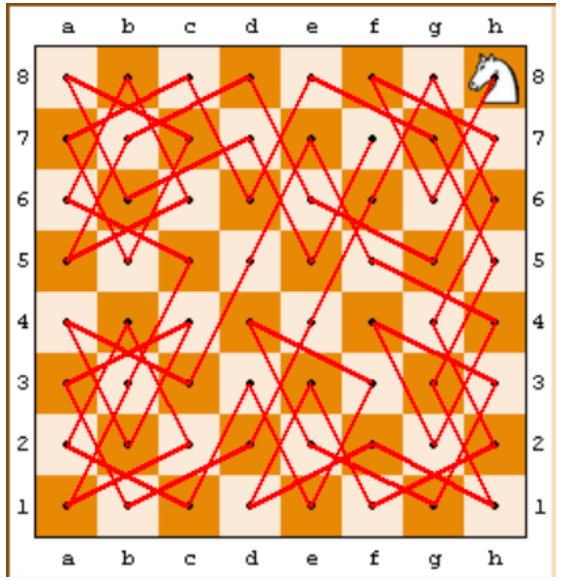
Jeu d’échecs : Trouver le meilleur coup

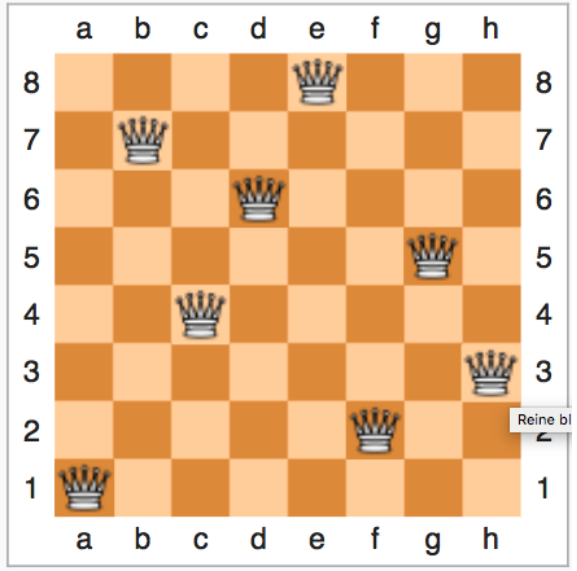
Problèmes d’échecs : Trouver une solution

- N reines : positionner N reines sur un échiquier *N x N*

- Déplacement de cavalier : chemin d’une position à une autre

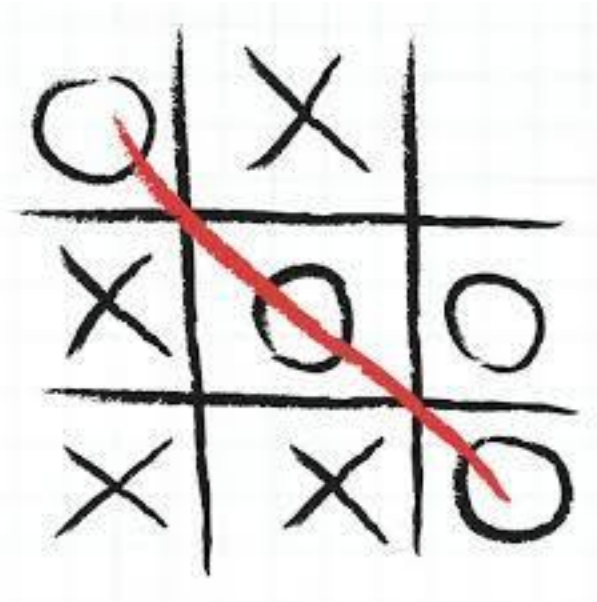
- Des exercices de *mat…*





Jeu du taquin (n-Puzzle) : Arranger les pièces

Jeu du morpion : avec adversaire



Jeux de cartes : partiellement observable

Plusieurs types de problèmes

- 1 joueur → trouver une séquence d'actions
- 2 ou plusieurs joueurs → meilleur coup, adversaires à objectifs contraires

- Déterministes → recherche exhaustive
- Recherche heuristique → algorithmes gloutons et optimisation locale
- Partiellement observable → algorithmes probabilistes
- Problèmes de satisfaisabilité → raisonnement logique (Prolog)

- Plein d'exemples disponibles ? → apprentissage automatique
- …

Jeux à 1 joueur

- Aspirateur, Echecs, N-Puzzle (taquin)

- Recherche d’un chemin minimum dans un graphe d’état
  - Poids positifs : Disjkstra ou Uniform-Cost Search

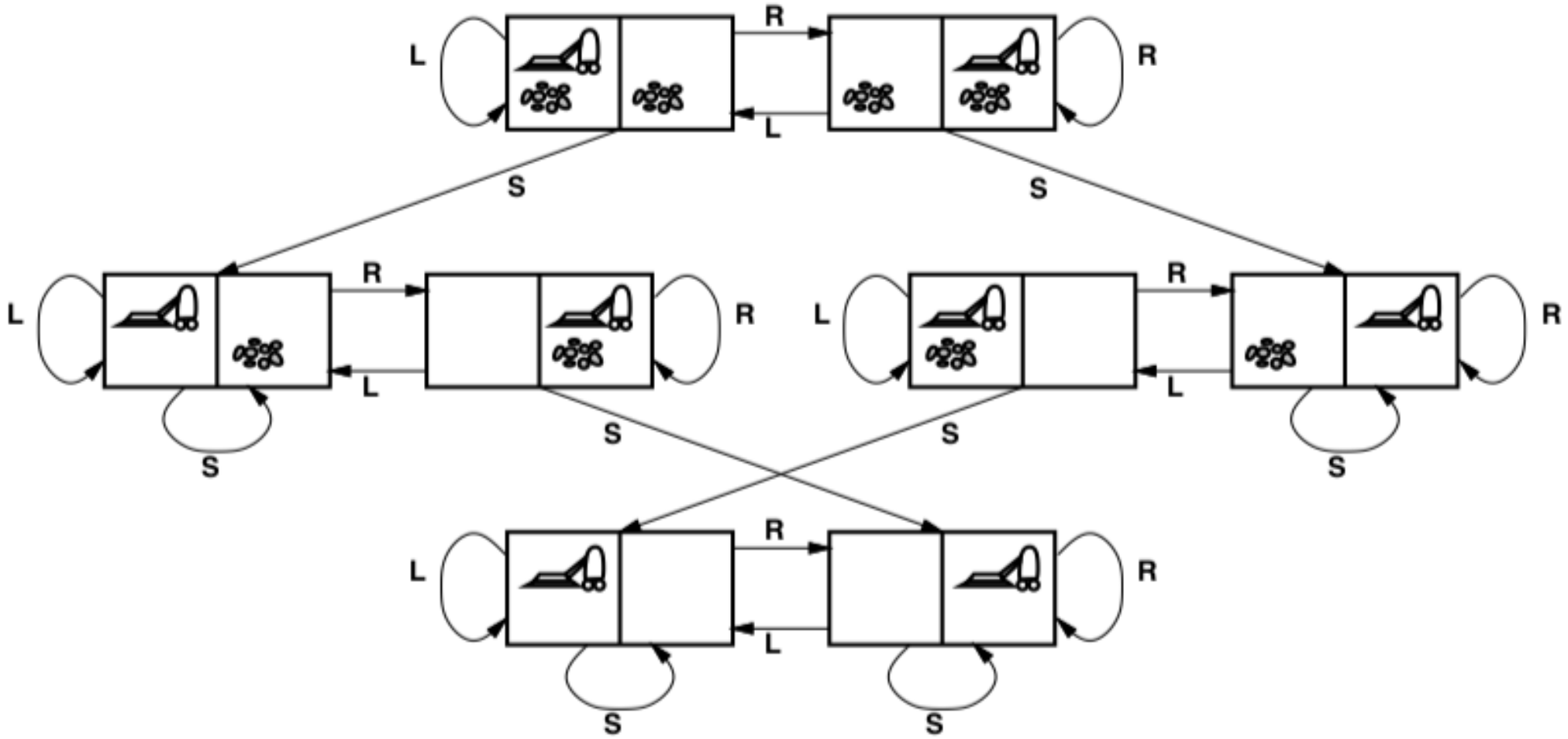
- Algorithmes de recherche classiques BFS, DFS

Le monde de l’aspirateur

- L’ancêtre de l’aspirateur autonome veut aspirer un appartement 2 cases

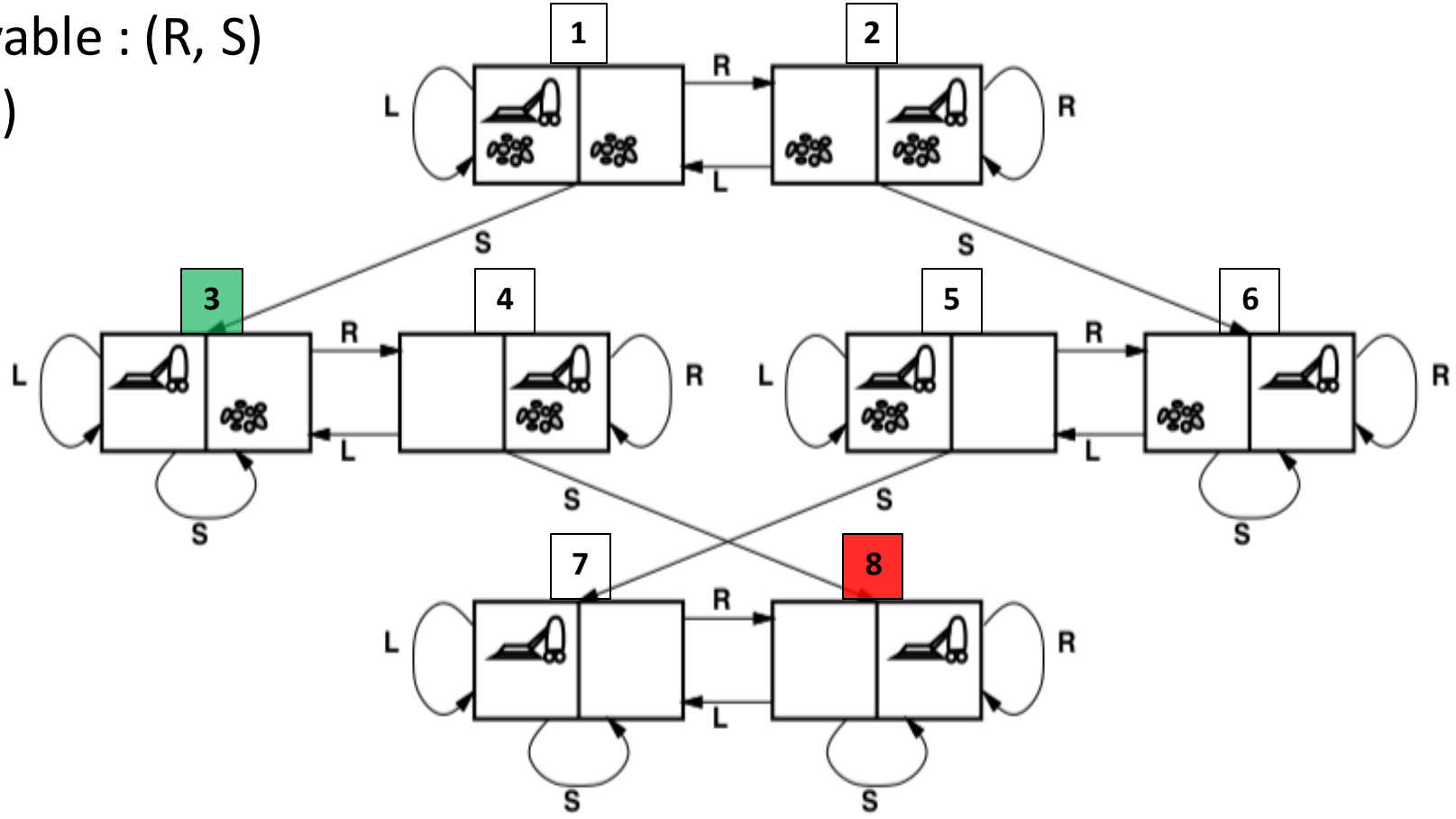
- Trois actions possibles
  - « Aller à gauche », « Aller droite », « Aspirer »
- Doté de capteurs qui ne fonctionnent pas toujours
- Représentation par graphe d’états
  - les sommets sont des états du monde
  - les arêtes sont des actions possibles

Le monde de l’aspirateur



Le monde de l'aspirateur

- Plus court chemin de l'état 3 à 8 ?
  - Complètement observable : (R, S)
  - Sans capteurs : (S, R, S)

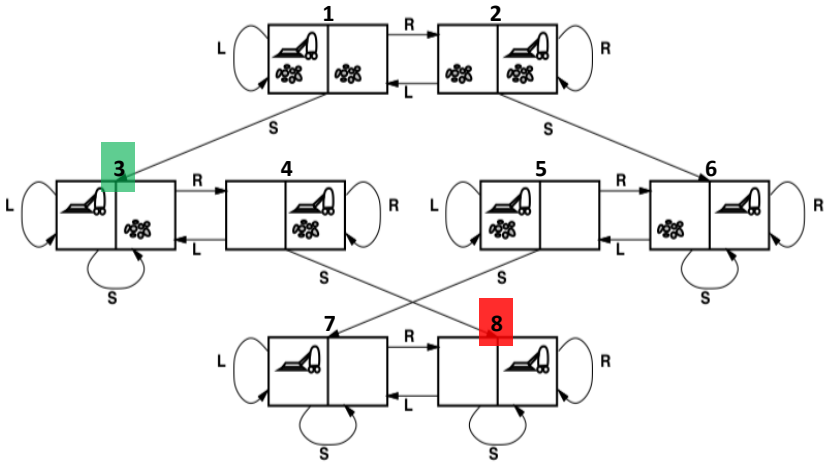


Modélisation d'un problème *Idée générale*

- **Simulation** des états et coûts des transitions dans un « arbre d'états »
  - Un état initial
  - Un état final → Test de terminaison ou fonction d'**évaluation**
  - Un ensemble d'**actions** (ex: tourner à droite ; aspirer)
  - Une fonction de **transition** : Etat x Action → Etat
    - souvent des règles connues (ex: impossible de dépasser les bords d'un échiquier)

- Une représentation du monde
  - tableau, matrice, arbre, graphe

- Une fonction de **coût** = poids/coût des transitions
  - distance, poids, heuristique, …



Algorithmes de recherche *Idée générale*

1. Démarrer la recherche avec la **liste** contenant l'*état initial* du problème

2. Si la liste n'est pas vide alors :

   (a) **Choisir** (à l'aide d'une **stratégie**) un état ***e*** à traiter.

   (b) Si ***e*** est un *état final* alors **retourner** recherche positive

   (c) Sinon, rajouter tous les **successeurs** de ***e*** à la liste d'états à traiter et recommencer au point 2.

3. Sinon **retourner** recherche négative.

Parcours d’arbres classiques

- Recherche non-informée

- Profondeur d’abord, Largeur d’abord : DFS, BFS
  - Stratégie possible d’ordonnancement des nœuds fils
  - Peu d’autres stratégies possibles…

- Eviter les cycles : mémoriser les états visités
  - Test d’appartenance → complexité minimale avec une table de hashage
  - « closed-list »

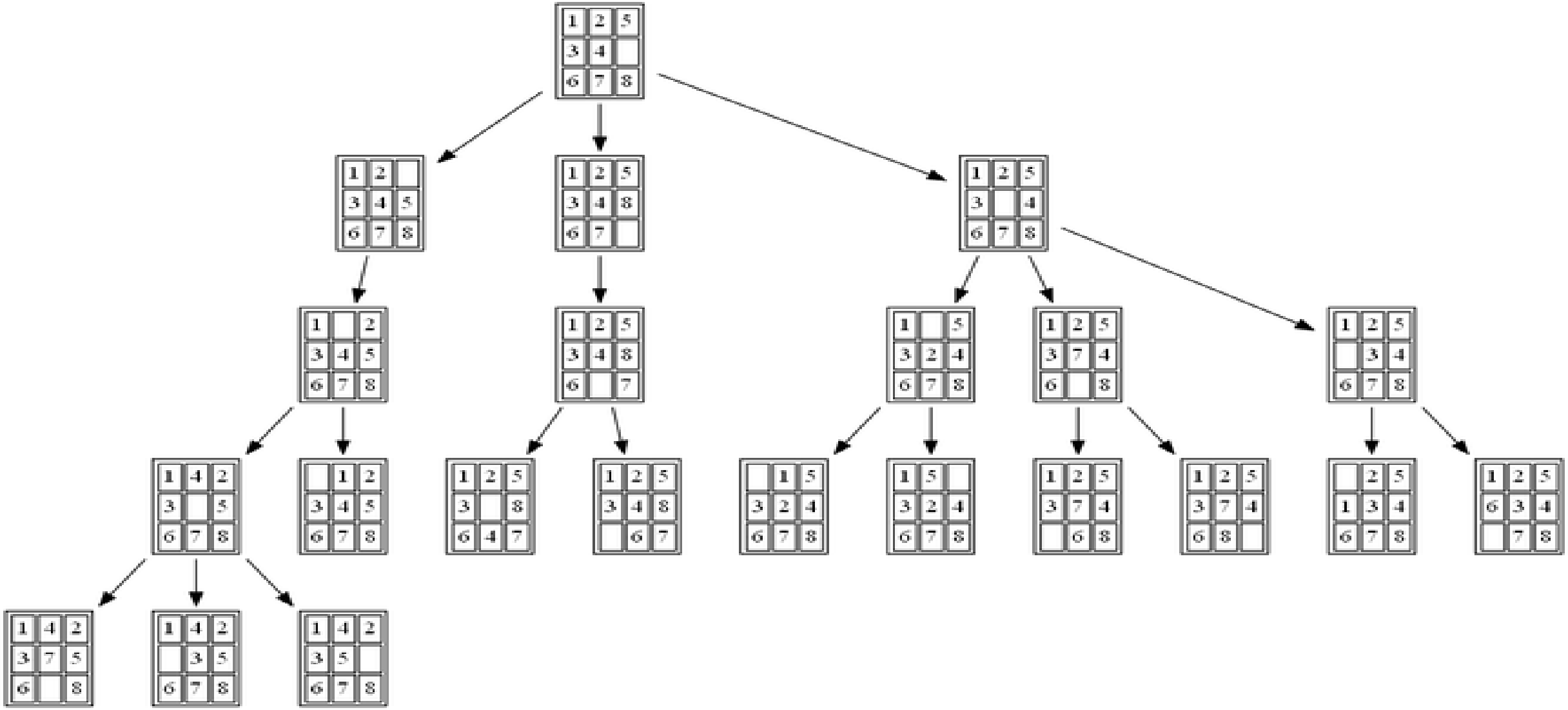
Modélisation d’un problème

Implémentation générale

- Une structure **Node** qui contient, au moins, un état ***STATE***, et un coût ***COST***

- Une fonction ***getActions(s)*** qui retourne les actions possibles à partir de l’état ***s***
- Une fonction ***makeState(s, a)*** retourne l’état issu de l’action ***a*** depuis l’état ***s***
- Une fonction ***getCost(s, a)*** retourne le coût (poids) de l’action ***a*** depuis l’état ***s***
- Une fonction ***isGoal(s)*** retourne vrai si ***s*** est l’objectif
- Une fonction ***makeChildNode(n, a)*** retourne un nouveau nœud en simulant une action ***a*** à partir du nœud ***n***

Exemple : n-Puzzle (jeu du taquin)



Algorithme de recherche avec coût : UCS

- Non informé, mais considère un coût par action/transition

- Uniform Cost Search
  - Coûts positifs
  - Utilise une liste avec priorité
  - Trouve une solution optimale = plus court chemin

- Cas particulier de Dijkstra
  - Trouve le chemin optimal entre l’état initial et l’état solution

Algorithme UCS

Pseudo-code général

```
UCS(): // returns a solution, or failure
    node ← makeNode(INITIAL-STATE)
    node.COST ← 0
    frontier.push(node) // priority queue with node as the only element
    explored ← an empty set
    while frontier is not empty do
        node ← frontier.pop()
        if problem.isGoal(node.STATE) then return SOLUTION(node) // SOLUTION() returns path from node to root
        if node.STATE is not in explored then
            explored.push(node.STATE) // avoid cycles
            for each action in getActions(node.STATE) do
              n ← makeChildNode(node, action)
              n.COST ← node.COST + getCost(node, action) // often included within makeChildNode()
              if n.STATE is not in explored (or in frontier) then
                  frontier.push(n) // replace node if exists with same STATE but COST greater
    end
```

Algorithme UCS

Remarques

- Basé sur une recherche en largeur d'abord
- L'objet *frontier* est une liste avec priorité
  - Complexité et optimalité dépendent de son implémentation
- Le coût de transition d'un état à un autre peut être constant
  - Par exemple dans le jeu du taquin = 1
- Les solutions sont une séquence d'actions entre le nœud racine et le nœud « solution »
  - Chemin = Fonction récursive qui boucle en suivant le *parent* d'un nœud
- Solution optimale s'il n'existe pas de solutions de coût inférieur

Les variantes d’algorithmes non informés

- Dépendent de la structure de liste utilisée
  - Pile → profondeur d’abord
  - File → largeur d’abord
  - Tas (priorisé par le coût) → Dijkstra (ou Uniform-Cost Search)

- Critère d’arrêt sur la profondeur maximale
  - Profondeur limitée = Depth-Limited-Search (DLS)
  - Profondeur itérative = Iterative-Deepening Search (IDS)

- Problèmes totalement observables
  - Ajout d’une fonction heuristique pour l’estimation du coût

  ➔ Algorithme A-star (A*)

Algorithme de recherche informé

- = « Recherche Heuristique »

- Exploite une connaissance du problème / monde
  - Exemple : où se trouve la cible ?

- Stratégies possibles pour la sélection de nœud à évaluer prioritairement
  - Coût d’un état (utilisé par UCS)
  - + Fonctions heuristiques

L’algorithme A*

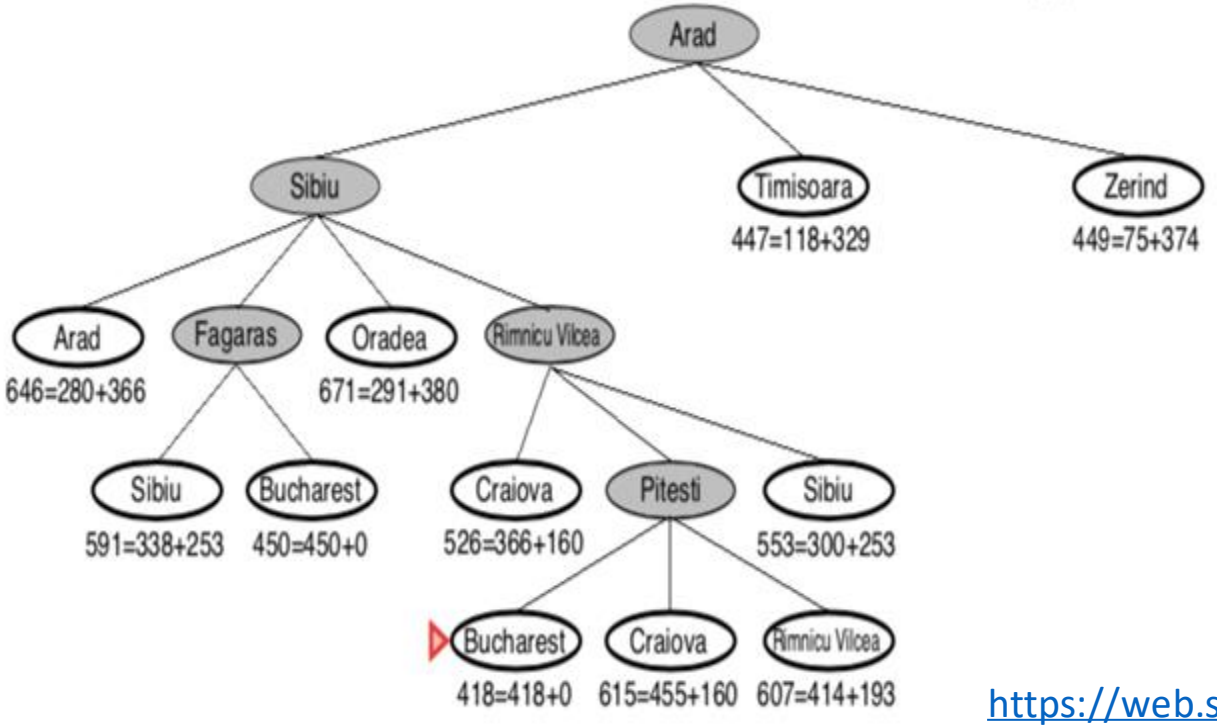
- Problème du plus court chemin / Recherche de solutions
- Guidé par une fonction heuristique, estimation de la distance au nœud cible
- [Nilsson et al., 1968]

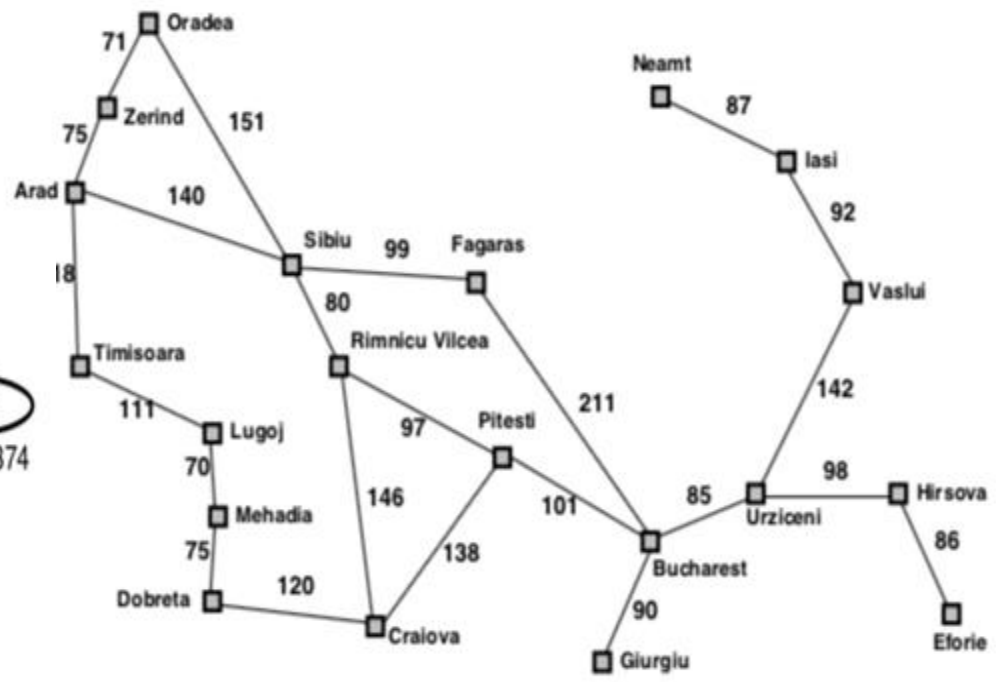
- Fonction de coût combine le coût d’exploration (classique) + heuristique
- f(n) = g(n) + h(n)
  - g(n) → coût pour aller de la racine au nœud ***n***
  - h(n) → estimation optimiste (minorante) du coût de ***n*** à la solution
  - f(n) → estimation du coût minimal pour aller de la racine à la solution en passant par ***n***

- UCS (ou Dijkstra) est un cas particulier de A* avec h(n) = 0

Exemple : chemin entre Arad et Bucarest

- Heuristique = distance à vol d'oiseau





Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1172/handouts/search.html

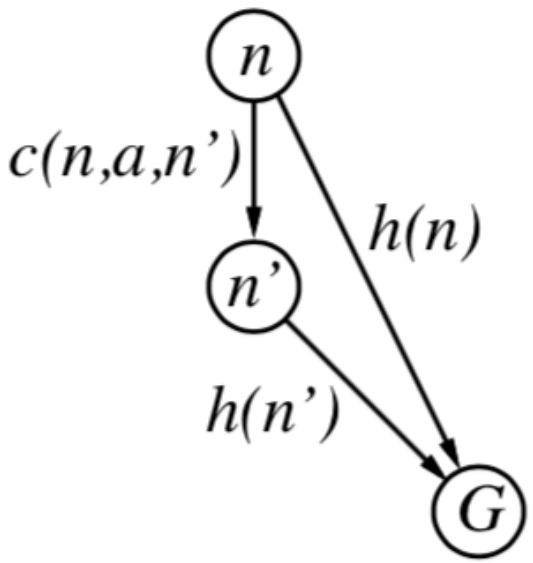
Exemple : n-Puzzle (jeu du taquin)

- Heuristique 1 = nombre de pièces mal placées
- Heuristique 2 = distance de Mahattan entre n.STATE et la solution

- Qu’est-ce qu’une fonction heuristique ?
- Quelle est la meilleure heuristique ?

Fonctions heuristiques

- Consistance
  - Si h(n) ≥ 0 et h(G) = 0 pour tout objectif G
  - h(n) ≤ cost(n, a, n’) + h(n’) pour tout fils n’ de n



- A* est optimal si h est consistante

- Exemple n-Puzzle
  - h1 et h2 sont consistantes
  - h2 ≥ h1 → h2 *domine* h1 ; h2 est une meilleure stratégie que h1

Bon d’accord… Quand est-ce qu’on joue ?

Bon d’accord… Quand est-ce qu’on joue ?

Ca dépend de votre efficacité en TP !

Les TPs

- TP1 + TP2 : Problèmes d’échecs
  - Structures Pile et File en listes chainées
  - 8 reines ; déplacements de cavalier
  - Comparaison DFS, BFS

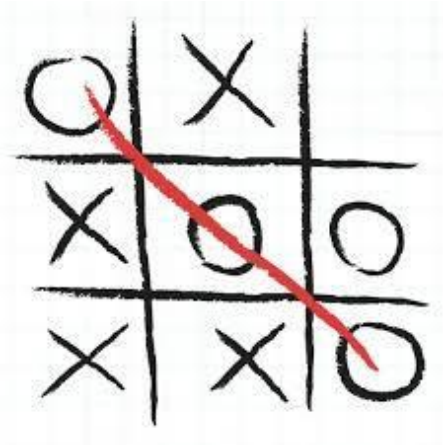
- TP3 + TP4 : Jeu du taquin
  - Liste de priorité
  - Comparaison UCS, A*

Jeux à plusieurs joueurs

- Joueurs adversaires, objectifs contraires
- Scénario typique : l’ordinateur joue contre un humain
- Stratégie optimale : algorithmes de recherche (actions déterministes)
- Apprentissage par renforcement / Apprentissage actif

Jeux et algorithmes de recherche

- Adapté aux jeux à somme nulle
  - « Le gain d'un joueur est la perte de l'adversaire »
- Beaucoup de jeux de plateaux : échecs, dames, morpion, awalé
- Un algorithme optimal existe : Min-Max
  - Basé sur le théorème du Minimax de von Neumann
  - Mais assez coûteux en temps et en espace
  - Des améliorations et approximations existent
- Exemple du morpion *N*x*N*



Morpion (tic-tac-toe)

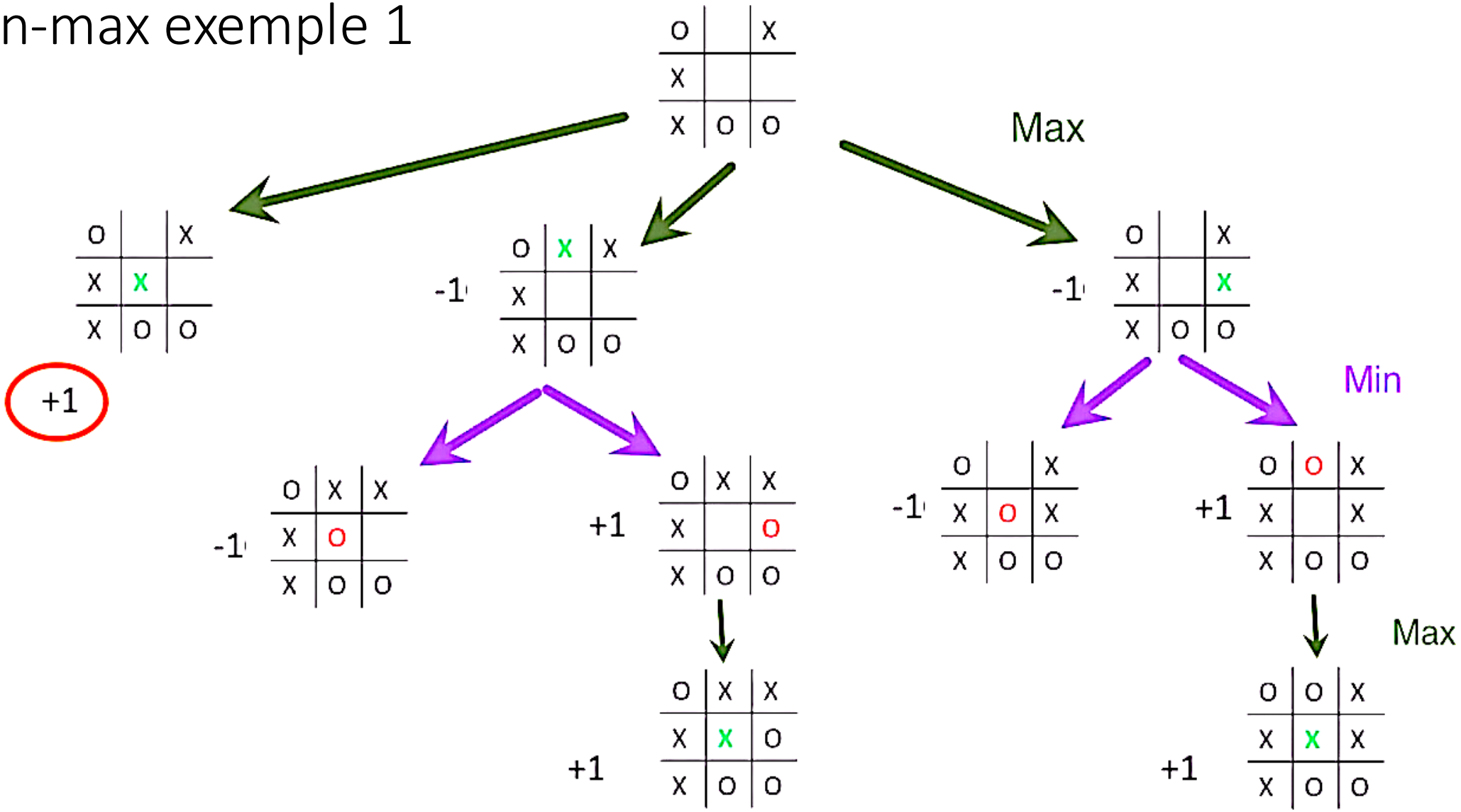
- 2 joueurs : « Max » et « Min »
  - Jouent à tour de rôle
  - Somme nulle

- État initial : planche 3x3 vide
- Etat final : une ligne, une colonne, ou une diagonale, remplie par un même joueur

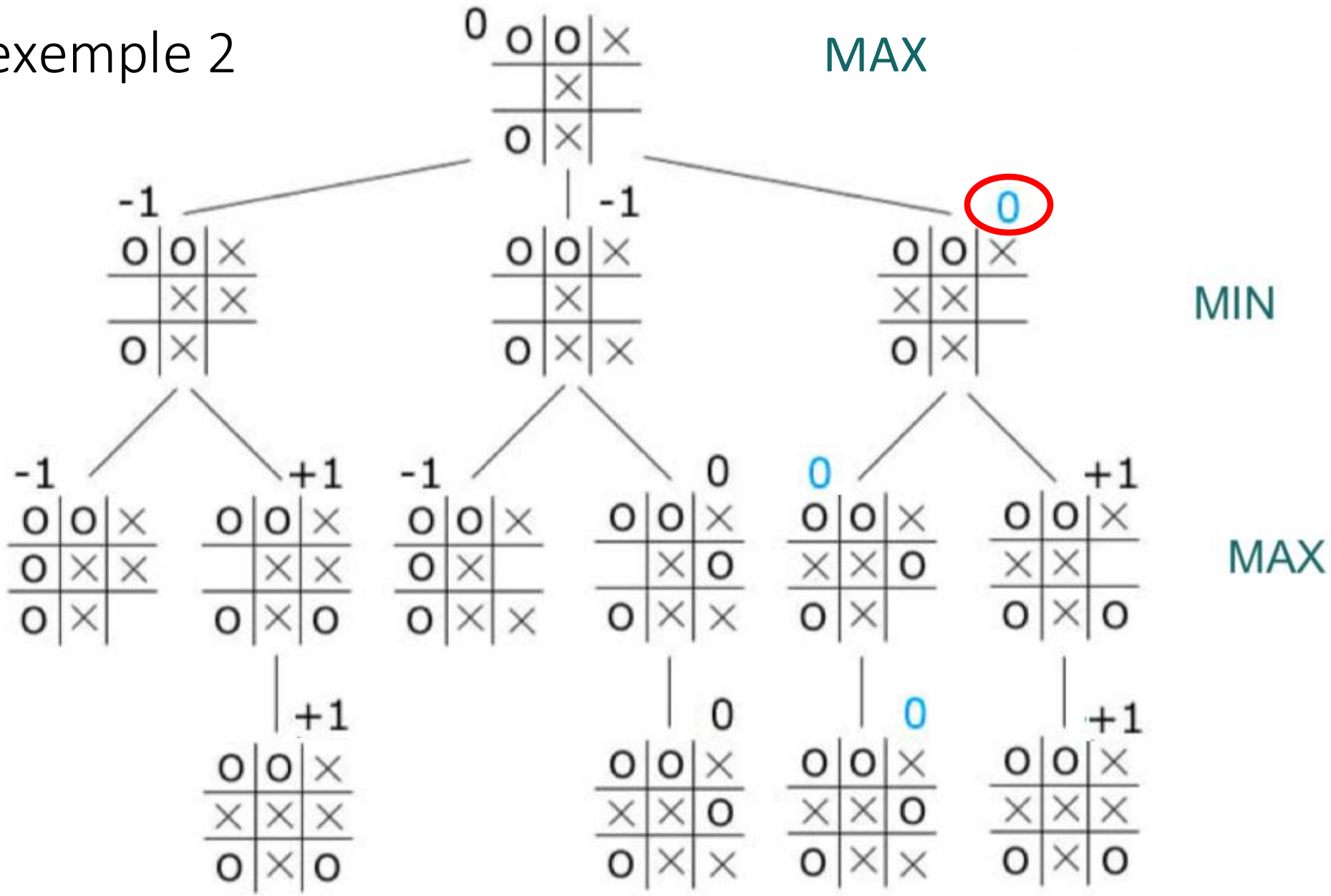
- Action : marquer une case vide

- Evaluation : +1 si « Max » gagne ; -1 si « Min » gagne ; 0 si égalité

Min-max exemple 1



Min-max exemple 2



Algorithme Minimax - *Idée générale*

- Donne le coup parfait pour un jeu déterministe à information parfaite
  - MAX doit choisir le coup qui maximise le minimum de ses gains ou qui minimise le maximum des pertes quand MIN joue

- Si *b* est le nombre de coups possibles par situation et *m* la profondeur maximale de l'arbre, minimax a une complexité :
  - en temps de $O(b^m)$
  - en espace O(bm)

- Jeu d'échec : b ≈ 35, m ≈ 100
  - 35000000000

Algorithme Minimax - *Pseudo-code*

```
minimax(node, depth, player):
    if node is a terminal node then
        return the heuristic value of node
    if player == MAX then
        value := −∞
        for each child of node do
            value := max(value, minimax(child, depth + 1, MIN))
        return value
    else /* minimizing player */
        value := +∞
        for each child of node do
            value := min(value, minimax(child, depth + 1, MAX))
        return value
```

- Appel initial → `minimax(initialState, 0, MAX)`

Algorithme Minimax - *Variantes*

- Temps limité → limiter la profondeur *(cut-off)*

- Ne pas tout explorer → élagage d'arbre *(alpha-beta)*

- Simplification algorithmique → *negamax*
  - $\max(a, b) = -\min(-a, -b)$

Algorithme negamax

```
negamax(node, depth, color):
    if node is a terminal node then
        return color × the heuristic value of node
    value := −∞
    for each child of node do
        value := max(value, −negamax(child, depth + 1, −color))
    return value
```

- Appel initial → `negamax(initialState, 0, 1)`
- Equivalent à l’algorithme minimax

Elagage alpha-beta - *Principe*

- On définit deux bornes : α et β
  - α correspond à la meilleure valeur que le joueur MAX peut garantir à ce niveau ou supérieur.
  - β est la meilleure valeur que MIN peut garantir à ce niveau ou supérieur.

- Au départ, $\alpha = -\infty$ et $\beta = +\infty$

- Les nœuds élagués sont ceux tels que $h(n) \in [\alpha, \beta]$ et $\alpha \geq \beta$

Elagage alpha-beta - Pseudo-code

```
function alphaBeta(node, depth, alpha, beta, maximizingPlayer):
  if depth == 0 or node is terminal:
    return heuristicValue(node)
  if maximizingPlayer:
    value = -∞
    for each child in children(node):
      value = max(value, alphaBeta(child, depth - 1, alpha, beta, False))
      alpha = max(alpha, value)
      if alpha ≥ beta:
        break   // Beta cutoff – prune remaining branches
    return value
  else:  // Minimizing player
    value = +∞
    for each child in children(node):
      value = min(value, alphaBeta(child, depth - 1, alpha, beta, True))
      beta = min(beta, value)
      if alpha ≥ beta:
        break   // Alpha cutoff – prune remaining branches
    return value
```

Elagage alpha-beta - Pseudo-code 2

- Version negamax avec élagage *alpha-beta*

```
negamax(node, depth, α, β, color):
    if node is a terminal node then
        return color × the heuristic value of node

    value := −∞
    foreach child of node do
        value := max(value, −negamax(child, depth − 1, −β, −α, −color))
        α := max(α, value)
        if α ≥ β then
          break // cut-off!
    return value
```

Elagage alpha-beta - Pseudo-code 3

- L'ordre des mouvements a une importance…

```
negamax(node, depth, α, β, color):
    if node is a terminal node then
        return color × the heuristic value of node

    childNodes := generateMoves(node)
    childNodes := orderMoves(childNodes) // sort by heuristic value!
    value := −∞
    foreach child in childNodes do
        value := max(value, −negamax(child, depth − 1, −β, −α, −color))
        α := max(α, value)
        if α ≥ β then
          break // cut-off!
    return value
```

Elagage alpha-beta - avec *transpositionTable*

```
negamax_alpha-beta(node, depth, α, β, color):
    alphaOrig := α
    ttEntry := transpositionTableLookup(node)
    if ttEntry is valid and ttEntry.depth ≤ depth then
        if ttEntry.flag = EXACT then return ttEntry.value
        else if ttEntry.flag = LOWERBOUND then
            α := max(α, ttEntry.value)
        else if ttEntry.flag = UPPERBOUND then
            β := min(β, ttEntry.value)
        if α ≥ β then return ttEntry.value

    if depth > MAX_DEPTH or node is a terminal node then
        return color × the heuristic value of node

    childNodes := generateMoves(node)
    childNodes := orderMoves(childNodes)
```

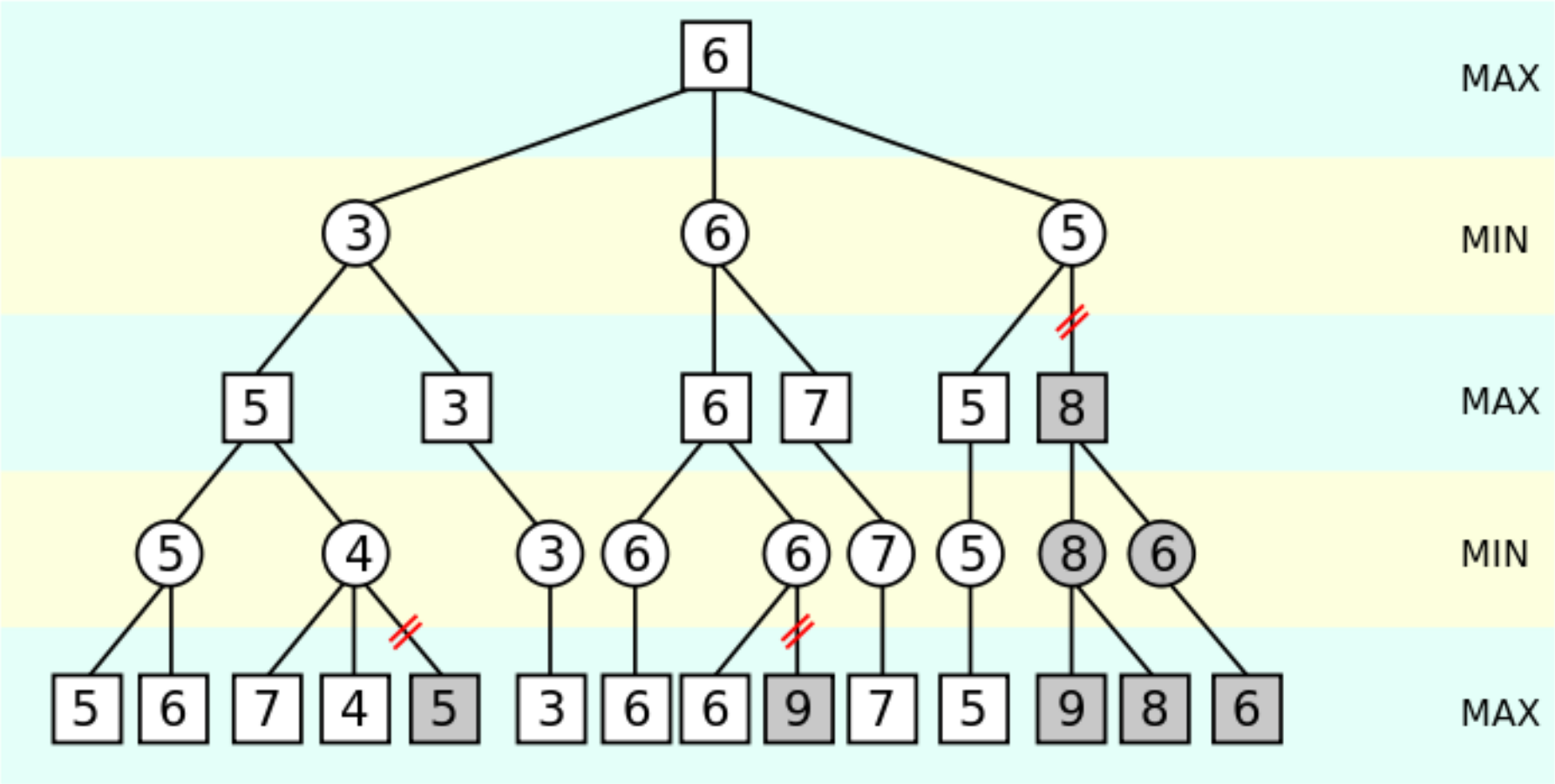
```
    value := -∞
    for each child in childNodes do
        value := max(value, -negamax(child, depth + 1, -β, -α, -color))
        α := max(α, value)
        if α ≥ β then  break

    ttEntry.value := value
    ttEntry.flag := EXACT
    if value ≤ alphaOrig then
        ttEntry.flag := UPPERBOUND
    else if value ≥ β then
        ttEntry.flag := LOWERBOUND
    ttEntry.depth := depth
    transpositionTableStore(node, ttEntry)

    return value
```

- Appel initial → `negamax_alpha-beta(initialState, 0, -∞, +∞, 1)`

Elagage alpha-beta - *Exemple*



Elagage alpha-beta - *Remarques*

- Algorithme très similaire à Minimax !

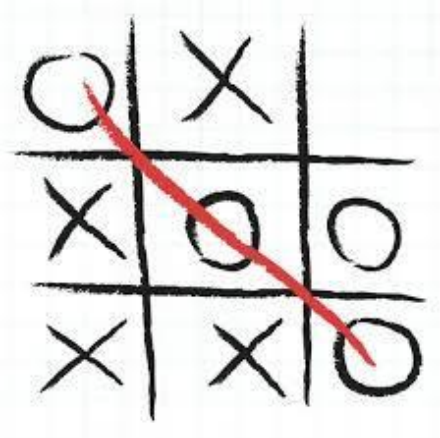
- Comme minimax, peut être amélioré : Iterative Deepening Search

- Efficace si l’arbre est bien ordonné
  ➔ Stratégie d’ordonnancement ?

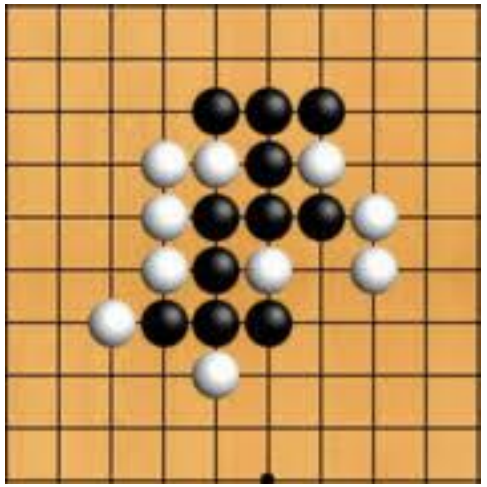
- Permet d’explorer à une profondeur bien supérieure à Minimax

Les TPs

- 2 séances sur l'implémentation du Morpion
  - Fonctions heuristiques et évaluation
  - Min-max
  - Alpha-beta



- 1 séance sur une extension : Gomoku
  - Fonctions heuristiques et évaluation



- 1 séance supplémentaire pour peaufiner le tout
  - Evaluation par contrôle continu + *incrément*
  - *Ajouter une interface graphique ?*